Student Learning Management System

High Level Design Document

Upgrade2Architect Phase 3 — Version 1.0

Technology Stack: AWS | Java Spring Boot | React | Docker | ECS Fargate

Document attribute	Value
Document title	Student LMS — High Level Design
Version	1.0
Status	Draft
Technology stack	AWS, Java Spring Boot, React/Next.js, Docker, ECS Fargate
Author	Pratyush Chandra Jha - 2154615
Date	22-May-2026
Classification	Confidential

Table of Contents

1. Introduction

1.1 Purpose

This High Level Design (HLD) document describes the architectural approach, system structure, technology decisions, and component interactions for the Student Learning Management System (LMS). It is intended to provide the Architecture Review Board (ARB) with a comprehensive understanding of how the system is designed to meet all functional and non-functional requirements.

1.2 Scope

The LMS platform will serve three user personas — Admin, Instructor, and Student — and will support the following core capabilities:

- Course lifecycle management: create, publish, approve, archive
- Video, PDF, and text content delivery at scale
- Assessment engine: timed quizzes, auto-grading, manual grading
- Student progress tracking and certificate generation
- Real-time and asynchronous notifications
- Event logging and audit trails
- Role-based access control (RBAC) with JWT authentication

1.3 Intended audience

This document is intended for the Architecture Review Board, engineering leads, senior developers, and infrastructure engineers involved in the delivery of the LMS platform.

1.4 Document references

Reference	Description
F001–F011	Functional requirements — UI, API, auth, user mgmt, courses, assessments, progress, grading, notifications, logging, code quality

Reference	Description
NFR001	Non-functional requirements — error handling, rate limiting, 30s load SLA, Docker, CI/CD
AWS Well-Architected Framework	Five pillars: operational excellence, security, reliability, performance efficiency, cost optimisation
Spring Boot 3.x	Java application framework for microservices
React / Next.js	Frontend framework for the SPA

2. System Overview

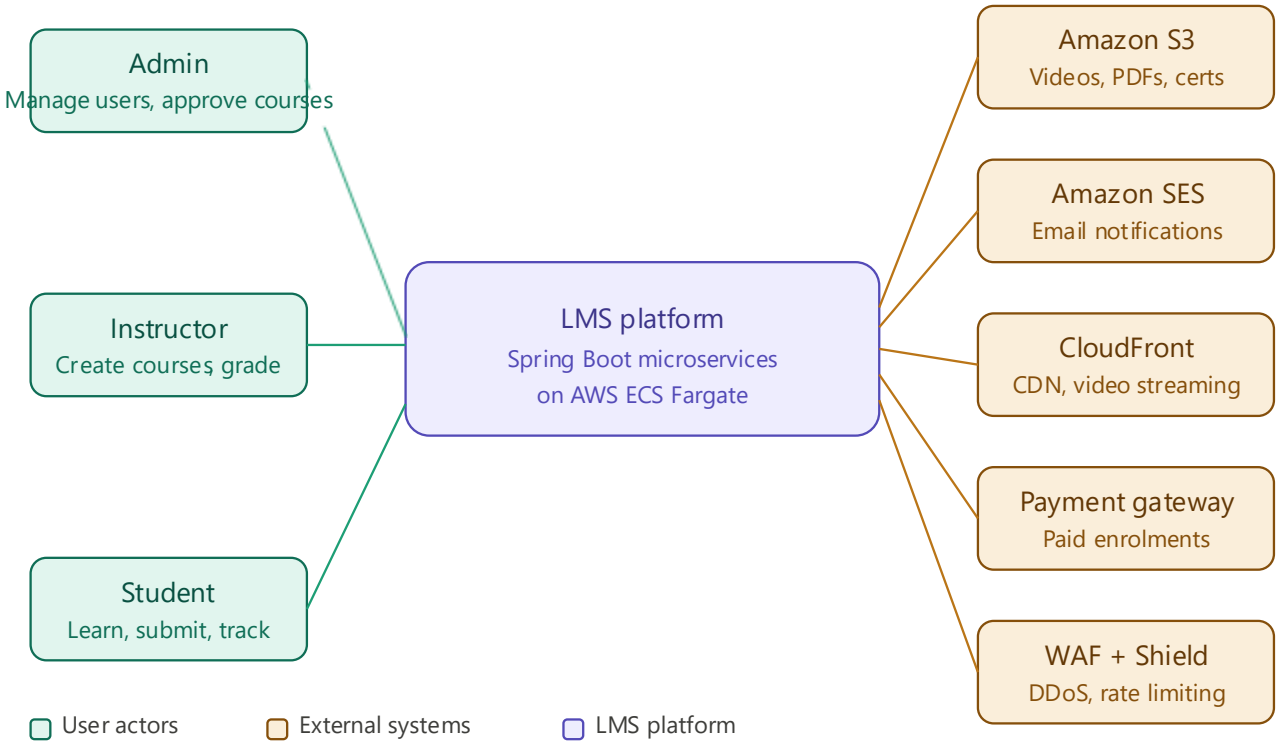
2.1 Problem statement

> Build a full-stack Student LMS web application that enables educational institutions to manage courses, content, assessments, and student progress in one platform.

The LMS must support three distinct user roles with different permission boundaries. Instructors create and publish structured course content including video lessons, assignments, and quizzes. Students enroll in courses, consume content, submit assessments, and track their progress through a dashboard. Admins manage users, approve course publications, and generate performance reports.

2.2 System context

The LMS interacts with the following external systems and actors:



Actor / System	Type	Interaction
Admin	User actor	Manage users, approve courses, view reports and logs
Instructor	User actor	Create/publish courses, grade submissions, post announcements
Student	User actor	Enrol, consume content, submit assessments, track progress
Amazon S3	External AWS service	Store and stream videos, PDFs, assignment uploads, certificates
Amazon SES	External AWS service	Deliver transactional email notifications
Amazon CloudFront	External AWS service	CDN delivery of static assets and video content
Payment Gateway	Third-party service	Process payments for paid course enrolments
AWS WAF + Shield	External AWS service	DDoS protection and rate limiting at the edge

2.3 Design principles

The following principles guided all architectural decisions in this document:

Principle	Application to LMS
Microservices decomposition	Eight independently deployable services, one per bounded domain context
Event-driven async processing	Heavy operations (grading, cert gen, notifications) processed asynchronously via SNS/SQS/Lambda

Principle	Application to LMS
Security by default	JWT auth at gateway, RBAC in every service, VPC isolation, IAM least-privilege, KMS encryption
Performance at edge	CloudFront CDN + S3 pre-signed URLs ensure 30s content load SLA (NFR001)
Operability	Structured logging, CloudWatch dashboards, X-Ray tracing, SonarQube quality gates
Scalability	ECS Fargate auto-scaling, DynamoDB on-demand, SQS buffering for write spikes
Infrastructure as code	All AWS resources provisioned via CDK/CloudFormation for reproducibility

2.4 Functional Requirements

The following table details all functional requirements (F001–F011) for the LMS platform, covering UI, API integration, authentication, user management, course management, assessments, progress tracking, grading, notifications, event logging, and code quality.

Req ID	UI Component	Requirements & Implementation Notes
F001	**Student Dashboard**	Display enrolled courses, upcoming deadlines, recent grades, and progress percentage per course. React component polls Progress and Assessment services at a configurable interval (default 60s, min 30s). Progress bars rendered client-side from completion % returned in API response.
F001	**Course Player**	HLS video player with chapter navigation, timestamped notes, and bookmarks. Video streamed from S3 via CloudFront pre-signed URLs (satisfies 30s NFR001 SLA). Notes and bookmarks persisted via POST /progress/notes and POST /progress/bookmarks; loaded on player init.
F001	**Assessment Interface**	Timed quiz interface with visible countdown; auto-submit fires on expiry via client-side setInterval synced to server-issued start timestamp. EventBridge backup Lambda triggers auto-submit if client fails. MaxAttempts enforced server-side by Assessment service before allowing a new attempt.
F001	**Search & Filter**	Search courses by title, instructor, or category. Filter by difficulty level and estimated duration. GET

Req ID	UI Component	Requirements & Implementation Notes
		/courses?search=&category=&difficulty=&maxDuration= served by Course service. Results cached in ElastiCache Redis (10-min TTL). Frontend debounces input 300ms before firing request.
F001	**Auto Refresh**	Progress and grade data refreshed at a configurable interval (default 60s, min 30s) via React useEffect + setInterval. WebSocket push from API Gateway triggers an immediate UI update on grade release, bypassing the polling cycle for time-sensitive events.

Req ID	API	Requirements & Implementation Notes
F002	**Video Storage API**	Integrate with Amazon S3 for course video uploading and streaming. Instructors upload via Course service which generates a pre-signed S3 PUT URL. Students stream via CloudFront pre-signed GET URLs served directly from S3, bypassing the API tier. Satisfies 30s content load SLA (NFR001).
F002	**Email Notification API**	Amazon SES triggered by Notification service for: enrolment confirmation, assignment deadline reminders (48hr advance), and grade release. Events consumed from SQS queue published by Enrolment, Assessment, and Assessment/Progress services via SNS fanout.
F002	**Response Handling**	All REST API responses return JSON. Course content endpoints return structured lesson metadata. Quiz result endpoints return per-question scores, correct answers, and total grade. Progress endpoints return completion percentage, lesson view history, and certificate URL where applicable. Standardised error response schema applied globally via @ControllerAdvice.
F002	**Caching**	Course metadata (course:{id}:metadata) and module list (course:{id}:modules) cached in ElastiCache Redis with 10-minute TTL. Cache invalidated on any course update or publish event. Reduces RDS read load and improves API P95 response time.

Req ID	Feature	Requirements & Implementation Notes
F003	**Secure Login & Signup**	Registration and login supported for all three user types: Admin, Instructor, and Student. Auth service exposes POST /auth/register and POST /auth/login. API Gateway validates JWT on every subsequent request before forwarding to downstream services.
F003	**Password Security**	Passwords hashed using BCrypt with salt rounds >= 12 before persisting to RDS PostgreSQL. Plain-text passwords are never stored or logged. Password reset flow uses time-limited signed tokens.

Req ID	Feature	Requirements & Implementation Notes
F003	**Session Management**	JWT-based authentication: access token TTL 15 minutes, refresh token TTL 7 days. Refresh tokens stored in ElastiCache Redis; invalidated on logout (token blacklist). Refresh token rotation enforced on every use.
F003	**Role-Based Access Control**	Three roles: Admin (full platform access), Instructor (own courses, grading, announcements), Student (enrolled content, own submissions, own profile). Role claim embedded in JWT; each microservice enforces RBAC independently before processing requests.

Req ID	Role	Requirements & Implementation Notes
F004	**Admin**	Create, update, and deactivate user accounts. Assign and change user roles. View all user profiles with search and filter. User service exposes CRUD endpoints under /admin/users accessible only to Admin role JWT.
F004	**Instructor**	Manage own courses (create, update, publish, archive). View list of students enrolled in own courses. Grade assignment submissions and provide written feedback. Access restricted to own course data; cross-instructor data access rejected at service layer.
F004	**Student**	Manage own profile (name, avatar, notification preferences). View enrolment history and status of all enrolled courses. Track grades per assessment and cumulative course grade. No access to other students' data; user ID validated against JWT subject claim on every request.

Req ID	Feature	Requirements & Implementation Notes
F005	**Course Lifecycle**	Instructors can create, update, publish, and archive courses. Approval workflow: Draft -> Pending -> Published -> Archived. Admin must approve before a course appears in the public catalogue. Course service manages all state transitions with event published to SNS on each status change.
F005	**Course Structure**	Courses consist of Modules; each Module contains one or more Lessons. Lesson content types supported: text, video (S3/CloudFront), and PDF (S3). Relational schema: Courses(1)->Modules(many)->Lessons(many). Course service owns CRUD for all three levels.
F005	**Course Metadata**	Each course includes: category, difficulty level (Beginner, Intermediate, Advanced), and estimated duration. Metadata cached in ElastiCache Redis (10-min TTL). Searchable and filterable via GET /courses query parameters.

Req ID	Feature	Requirements & Implementation Notes
F005	**Enrolment**	Students can enrol in free courses directly. Paid course enrolment triggers payment verification via Payment Gateway integration in Enrolment service. On successful enrolment, an event is published to SNS which triggers an email confirmation via SES.

Req ID	Feature	Requirements & Implementation Notes
F006	**Question Types**	Instructors create quizzes with three question types: multiple choice, true/false, and short answer. Assessment service stores question bank per course. Question type determines grading path: objective types auto-graded; short answer queued for instructor review.
F006	**Timed Quizzes**	Each quiz has a configurable duration per attempt. Client-side countdown synced to server-issued start timestamp. Auto-submit triggered on expiry by React client; EventBridge Lambda provides backup trigger if client fails to submit within the deadline window.
F006	**Grading**	MCQ and true/false questions auto-graded by Assessment service Lambda on submission. Short answer submissions flagged as pending and appear in Instructor grading queue. Instructor grades via POST /submissions/{id}/grade with score and written feedback.
F006	**Retake Attempts**	Configurable MaxAttempts per quiz set by Instructor at quiz creation. Assessment service validates attempt count before allowing a new submission. Highest or latest score policy configurable per quiz.
F006	**Assignment Submissions**	File upload support for PDF, DOCX, and image formats. Student uploads via pre-signed S3 PUT URL issued by Assessment service. File metadata (S3 key, filename, upload timestamp) stored in RDS Submissions table. Instructor downloads via pre-signed S3 GET URL.

Req ID	Feature	Requirements & Implementation Notes
F007	**Completion Tracking**	Progress service tracks completion percentage per course and per module. Completion % = (lessons viewed + assessments submitted) / total items. Stored in RDS Enrollments table (courseCompletion, moduleCompletion columns). Lesson view events streamed via Kinesis -> Lambda -> DynamoDB for high write throughput.
F007	**Activity Recording**	Every lesson view, quiz attempt, and assignment submission fires an event to Kinesis Data Streams. Lambda consumer persists events to DynamoDB (Logs table) and updates Progress in RDS. Provides full audit trail and input data for cohort reports.

Req ID	Feature	Requirements & Implementation Notes
F007	**Certificate Generation**	When course completion reaches 100%, Progress service publishes a CertificateRequired event to SNS. Lambda subscriber generates a PDF certificate (student name, course title, completion date), stores it in S3, and updates the Enrollment record with the certificate URL. Student receives in-app and email notification with download link. Target SLA: < 30 seconds.
F007	**Cohort Reports**	Instructors can view progress reports for all students enrolled in their courses via GET /courses/{id}/progress/cohort. Admins can view cross-course cohort reports. Aggregate queries run against RDS read replica; summary data cached in ElastiCache for dashboard performance.

Req ID	Feature	Requirements & Implementation Notes
F008	**Grade Display**	Students view per-assessment scores and a cumulative course grade on their dashboard. Cumulative grade = weighted average of all graded assessments in the course. Assessment service exposes GET /students/{id}/grades and GET /enrollments/{id}/grade.
F008	**Written Feedback**	Instructors provide written feedback when grading short-answer questions and assignment submissions. Feedback stored in RDS Submissions table (feedback column). Visible to the student via their submission detail view once grade is released.
F008	**Grade Release Notifications**	On grade release (PATCH /submissions/{id}/release), Assessment service publishes a GradeReleased event to SNS. Notification service fans out to: SES (email to student) and API Gateway WebSocket (in-app push). Student sees grade notification in real-time without waiting for next polling cycle.
F008	**Admin Grade Reports**	Admins access grade distribution reports per course via GET /admin/courses/{id}/grade-distribution. Reports show score buckets (0-49, 50-69, 70-84, 85-100), mean, median, and pass rate. Queries run against RDS read replica; results cached for 10 minutes.

Req ID	Feature	Requirements & Implementation Notes
F009	**Notification Triggers**	In-app (WebSocket) and email (SES) notifications fired for: enrolment confirmation, new course content published, upcoming assignment deadline (48-hour advance warning via EventBridge Scheduler), and grade release. All triggers published as SNS events and consumed by Notification service.

Req ID	Feature	Requirements & Implementation Notes
F009	**Course Announcements**	Instructors post announcements via POST /courses/{id}/announcements. Announcement creation publishes an event to SNS; Notification service fans out to all enrolled students via WebSocket (in-app) and SES (email) based on their per-course preferences.
F009	**Student Preferences**	Students configure notification preferences per course (email on/off, in-app on/off) via PATCH /students/{id}/preferences. Preferences stored in ElastiCache Redis (TTL 60 minutes) and persisted to RDS. Notification service checks preferences before dispatching each notification type.

Req ID	Feature	Requirements & Implementation Notes
F010	**Logged Events**	All platform events logged: login, enrolment, lesson views, quiz attempts, assignment submissions, and application errors. Every microservice publishes log events to Kinesis Data Streams; Lambda consumer persists to DynamoDB (Logs table) and indexes in OpenSearch for querying.
F010	**Admin Log Viewer**	Admins filter logs by event type, actor (user ID or name), and date range via GET /admin/logs. OpenSearch powers full-text and range queries for fast filtering across large log volumes. Results paginated (default page size 50).
F010	**Log Structure**	Each log entry must include: actorId (user who triggered the event), eventType (LOGIN, ENROL, LESSON_VIEW, etc.), description (human-readable summary), and timestamp (ISO 8601 UTC). DynamoDB TTL configured for automatic log expiry per data retention policy.

Req ID	Feature	Requirements & Implementation Notes
F011	**Static Analysis**	SonarQube enforced on all Java/Spring Boot microservices: 0 blocker issues, 0 critical issues, coverage >= 80%. ESLint enforced on all React/TypeScript frontend code. Both gates run in CodeBuild pipeline; build fails if gate is not met (warning-only for first 2 sprints).
F011	**Coverage Threshold**	API code coverage must exceed 80% as measured by JaCoCo integrated with SonarQube. Coverage gate enforced in CI pipeline; promotion to staging blocked if threshold not met.
F011	**Mandatory Unit Tests**	Unit tests are mandatory for three critical logic areas: (1) Quiz grading logic — all question types and edge cases (unanswered, partial credit). (2) Progress calculation — completion % formula, 100%

Req ID	Feature	Requirements & Implementation Notes
		trigger for certificate generation. (3) Certificate generation trigger — event published correctly on completion, idempotency check. Implemented with JUnit 5 + Mockito; Amazon Q Developer used to accelerate test generation.

2.5 Non-Functional Requirements

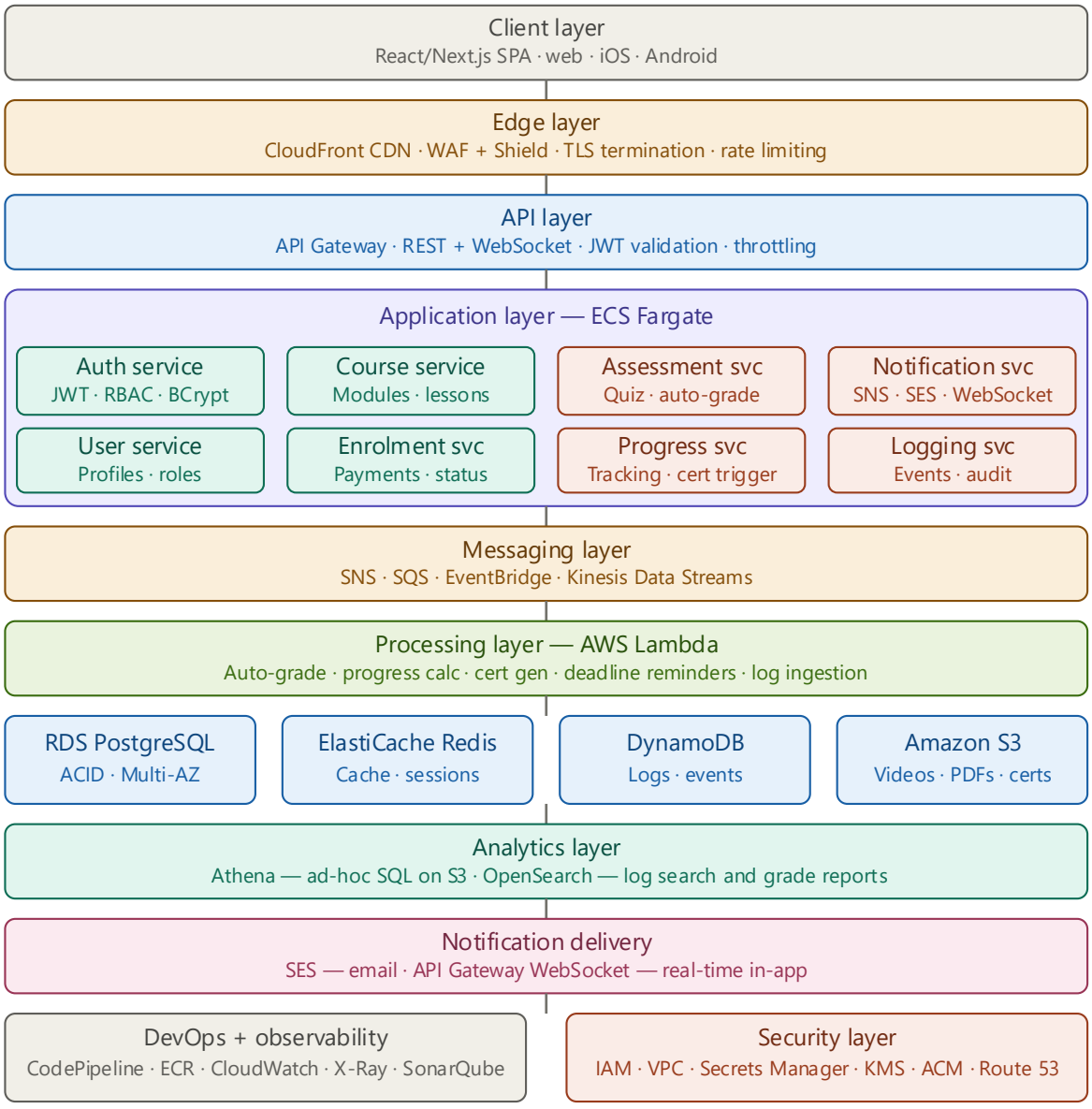
The following table details NFR001, the non-functional requirements governing error handling, rate limiting, performance SLAs, containerisation, and CI/CD for the LMS platform.

Req ID	Requirement	Design Decision & Implementation
NFR001	**Error / Exception Handling**	Comprehensive error handling throughout the application. Each Spring Boot microservice implements a global @ControllerAdvice exception handler returning a standardised JSON error schema (timestamp, status, errorCode, message, traceId). CloudWatch Alarms trigger on error rate thresholds; X-Ray traces capture all exceptions for root-cause analysis.
NFR001	**Rate Limiting**	API abuse prevention at two layers: (1) API Gateway throttling with configurable burst and steady-state limits per endpoint. (2) AWS WAF rate-based rules per source IP. Clients exceeding limits receive HTTP 429 with a Retry-After header. Limits tunable per environment without code changes.
NFR001	**Content Load SLA**	Course content (video, PDF, text) must load within 30 seconds. CloudFront CDN serves static assets from edge locations; S3 pre-signed URLs enable direct browser-to-CDN video streaming, bypassing the application tier entirely. ElastiCache Redis metadata cache reduces API round-trips. CloudFront P95 measured at < 2s for assets.
NFR001	**Containerised Deployment**	All eight Spring Boot microservices are containerised using Docker. Images built and tagged with Git SHA in AWS CodeBuild, pushed to Amazon ECR, and deployed to ECS Fargate with health checks. No EC2 instance management required; Fargate provides serverless container execution.
NFR001	**Modular Codebase & CI/CD**	Microservices architecture enforces modularity — each service has an independent CodePipeline. Pipeline stages: source -> lint/static analysis -> unit tests -> integration tests -> build -> image push -> quality gate -> staging deploy -> smoke tests -> production deploy (blue/green with automatic rollback). SonarQube quality gate and 80% coverage threshold enforced before any promotion.

3. Architecture Overview

3.1 Architecture style

The LMS adopts a microservices architecture deployed on AWS using containerised Spring Boot services orchestrated by Amazon ECS Fargate. The frontend is a React/Next.js single-page application served via CloudFront. Services communicate synchronously over REST (via API Gateway) for real-time operations and asynchronously over SNS/SQS for event-driven workflows such as grading, notifications, and certificate generation.



Teal = domain services · Coral = async processors · Amber = edge/messaging · Blue = data · Green = Lambda

3.2 Architecture layers

Layer	Components	Responsibility
Client layer	React/Next.js SPA (web, iOS, Android)	Responsive UI for all three user roles; auto-refresh, video player, quiz interface
Edge layer	Amazon CloudFront, AWS WAF + Shield	CDN delivery, DDoS protection, rate limiting, TLS termination
API layer	Amazon API Gateway (REST + WebSocket)	Single entry point, JWT validation, throttling, WebSocket for real-time notifications
Application layer	8 x ECS Fargate Spring Boot microservices	Business logic execution; Auth, User, Course, Enrolment, Assessment, Progress, Notification, Logging
Messaging layer	SNS, SQS, EventBridge, Kinesis	Async decoupling; event fanout, task queues, scheduled triggers, log ingestion
Processing layer	AWS Lambda	Serverless async processors: auto-grading, progress calc, cert gen, deadline reminders
Data layer	RDS PostgreSQL, ElastiCache Redis, DynamoDB, S3	Persistent storage, caching, event store, object store
Analytics layer	Amazon Athena, OpenSearch	Ad-hoc log queries, grade reports, log search and filtering
Notification delivery	SES, API Gateway WebSocket	Email delivery, real-time in-app push notifications
Observability layer	CloudWatch, X-Ray	Metrics, logs, distributed tracing, alarms
DevOps layer	CodePipeline, CodeBuild, ECR, SonarQube	CI/CD pipeline, Docker image registry, code quality gates

Layer	Components	Responsibility
Security layer	IAM, VPC, Secrets Manager, KMS, ACM, Route 53	Identity, network isolation, secrets, encryption, DNS, TLS

4. Microservice Design

4.1 Service catalogue

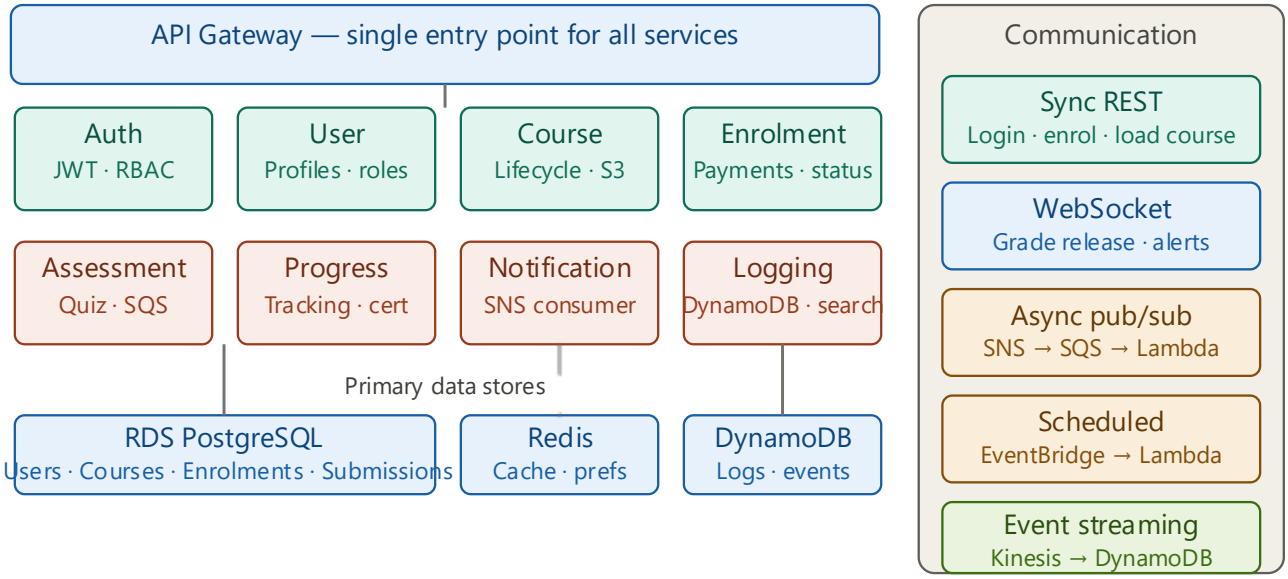
The LMS is decomposed into eight microservices, each owning its domain data and exposing REST APIs via the API Gateway. Services communicate asynchronously via SNS/SQS for cross-domain events.

Service	Domain	Key responsibilities	Primary data store
Auth service	Identity & security	Register/login/logout, JWT issuance and validation, RBAC enforcement, password hashing (BCrypt)	RDS PostgreSQL (Users table)
User service	User management	CRUD for user profiles, role assignment, deactivation (Admin), profile updates (Student)	RDS PostgreSQL (Users table)
Course service	Content management	Course/module/lesson CRUD, approval workflow (Draft→Pending→Published→Archived), category and difficulty management	RDS PostgreSQL (Courses, Modules, Lessons)
Enrolment service	Enrolment & payments	Enrol/unenrol students, payment verification for paid courses, enrolment confirmation event	RDS PostgreSQL (Enrollments)
Assessment service	Quizzes & assignments	Quiz/assignment CRUD, attempt management, MaxAttempts enforcement, file upload coordination	RDS PostgreSQL (Assessments, Submissions)
Progress service	Learning analytics	Track lesson views and completion, compute completion %, trigger certificate generation at 100%	RDS PostgreSQL (Enrollments), DynamoDB (events)

Service	Domain	Key responsibilities	Primary data store
Notification service	Communications	Consume grade/enrolment/deadline events, fan out to SES (email) and WebSocket (in-app), honour student notification preferences	ElastiCache Redis (preferences)
Logging service	Audit & observability	Receive and persist all platform events, expose filtered log query API for Admin	DynamoDB (Logs), OpenSearch (index)

4.2 Service communication patterns

Pattern	Used for	AWS services
Synchronous REST	User-facing real-time operations: login, enrol, load course, submit quiz	API Gateway → ECS Fargate
WebSocket	Real-time in-app notifications (grade release, announcements)	API Gateway WebSocket → Notification service
Async publish/subscribe	Grade release → notify student; enrolment → send confirmation email	SNS → SQS → Lambda / Notification service
Scheduled events	48-hour deadline reminders; auto-refresh triggers	EventBridge Scheduler → Lambda
Event streaming	High-volume lesson view and log events — buffered ingestion	Kinesis Data Streams → Lambda → DynamoDB



Teal = synchronous domain services · Coral = async processing services

5. Data Architecture

5.1 Data store selection rationale

Data store	AWS service	Rationale
Relational DB	Amazon RDS PostgreSQL (Multi-AZ)	ACID transactions for users, courses, enrolments, assessments; complex joins for reporting; foreign key integrity
Cache	ElastiCache Redis	10-minute course metadata cache (F002); JWT session store; notification preference store — sub-millisecond read latency
NoSQL document store	Amazon DynamoDB (on-demand)	High write-throughput for logs and progress events; no fixed schema; auto-scaling; TTL for log retention
Object store	Amazon S3	Video storage and streaming via CloudFront; PDF/DOCX assignment uploads; generated PDF certificates; static frontend assets
Search index	Amazon OpenSearch	Full-text and range queries for Admin log filtering (event type, actor, date range); grade distribution reports
Analytics	Amazon Athena	Ad-hoc SQL queries on S3-archived log data for reporting without ETL

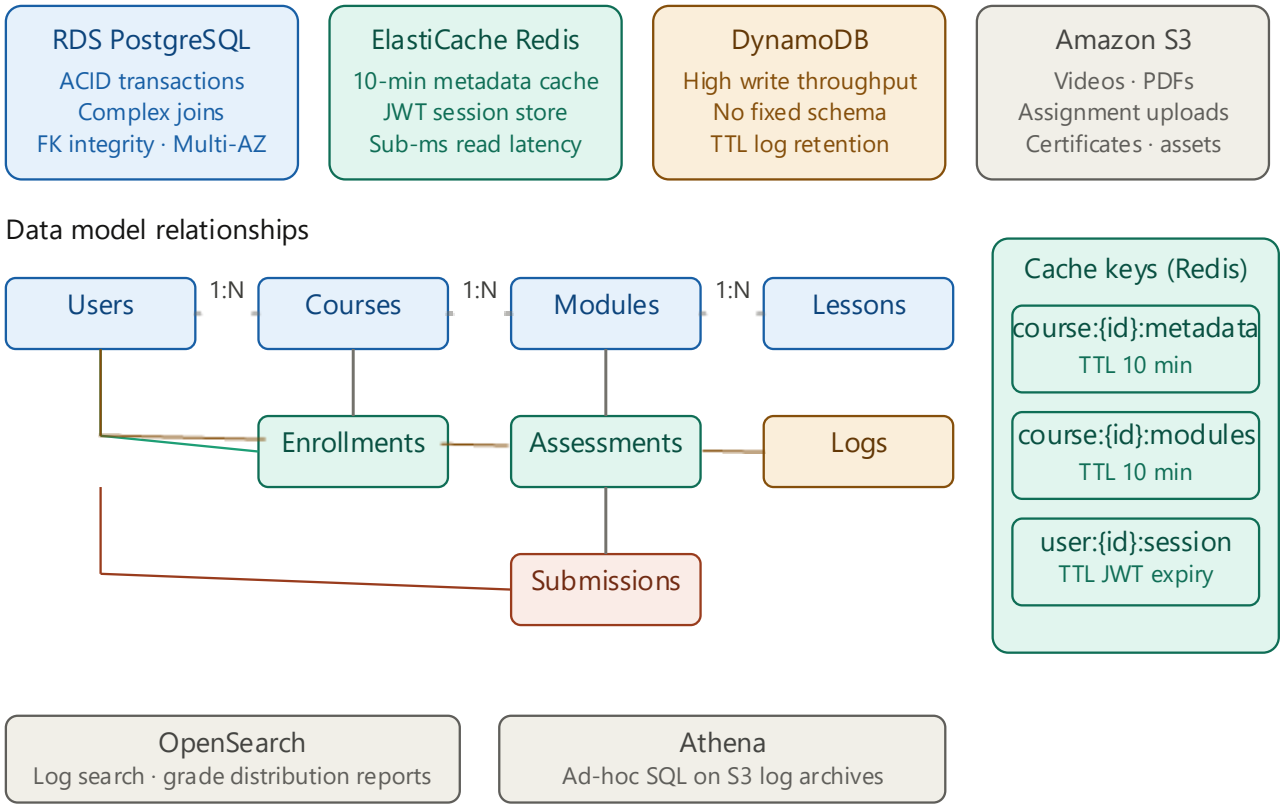
5.2 Data model summary

The core relational schema consists of seven tables with the following key relationships:

- Users (1) → (many) Courses via InstructorId (FK)
- Courses (1) → (many) Modules via CourseId (FK)
- Modules (1) → (many) Lessons via ModuleId (FK)
- Courses (1) → (many) Assessments via CourseId (FK)
- Users (1) → (many) Enrollments via StudentId (FK)
- Courses (1) → (many) Enrollments via CourseId (FK)
- Assessments (1) → (many) Submissions via AssessmentId (FK)
- Users (1) → (many) Submissions via StudentId (FK)
- Users (1) → (many) Logs via ActorId (FK)

5.3 Caching strategy

Cache key	TTL	Invalidation trigger
course:{id}:metadata	10 minutes	Course update or publish event
course:{id}:modules	10 minutes	Module add/update/delete
user:{id}:session	JWT expiry	Logout or token refresh
user:{id}:preferences	60 minutes	Notification preference update
assessments:{courseId}	10 minutes	Assessment add/update/delete



6. Security Architecture

6.1 Authentication and authorisation

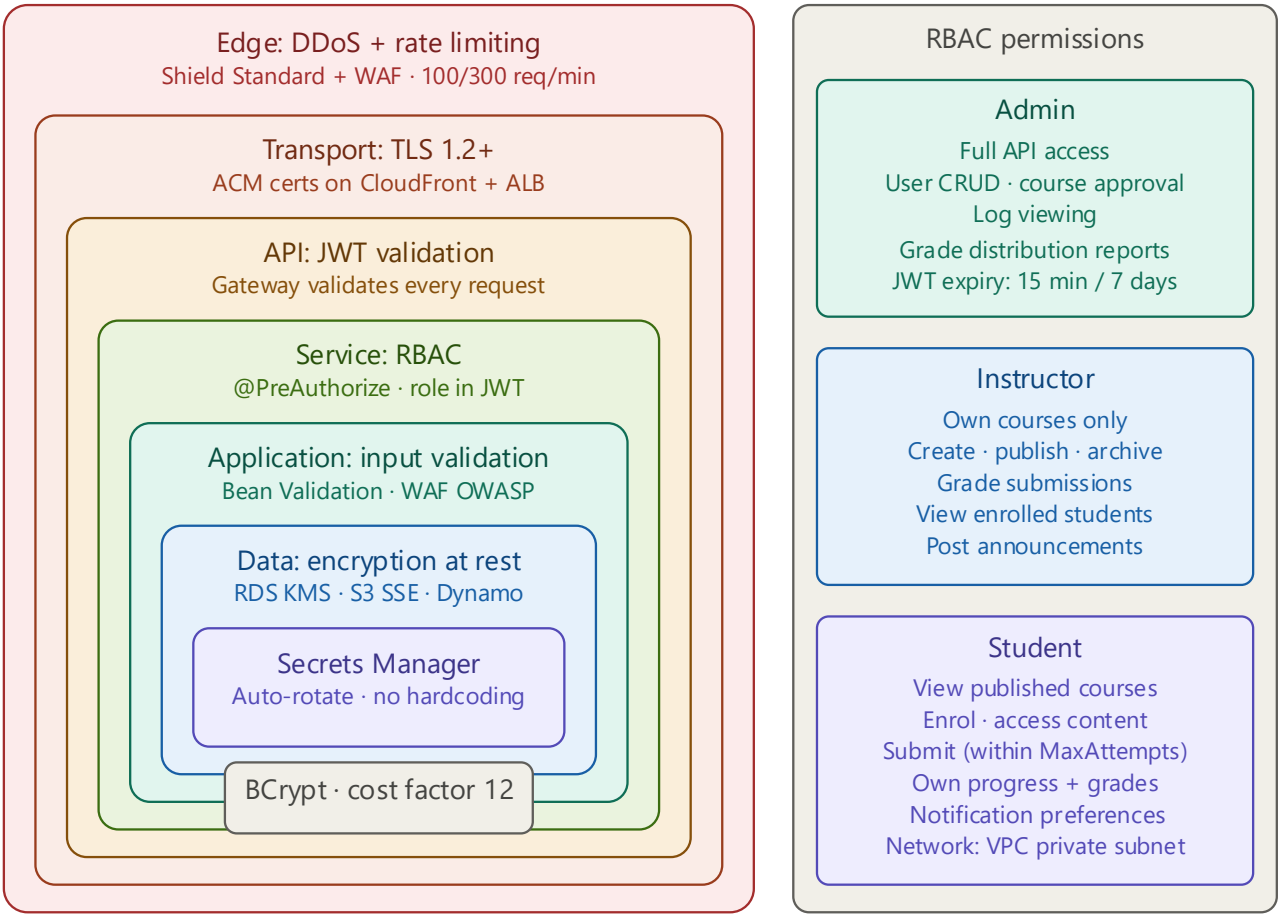
All API requests must carry a valid JWT issued by the Auth service. The API Gateway validates the token signature before forwarding requests. Each microservice performs RBAC checks against the role claim embedded in the JWT. Three roles are supported:

Role	Permitted operations
Admin	Full access to all APIs including user management (CRUD), course approval, log viewing, and grade distribution reports
Instructor	Create/update/publish/archive own courses; create assessments; grade submissions; view enrolled students; post announcements
Student	View published courses; enrol; access own enrolled content; submit assessments (within MaxAttempts); view own progress and grades; manage own notification preferences

6.2 Security controls

Control	Implementation	Requirement addressed
Password hashing	BCrypt with salt rounds ≥ 12	F003
JWT expiry	Access token: 15 minutes; Refresh token: 7 days	F003
Rate limiting	API Gateway throttling + AWS WAF rules (per IP and per user)	NFR001
Transport encryption	TLS 1.2+ via ACM certificates on CloudFront and ALB	NFR001
Data encryption at rest	RDS: AES-256 via KMS; S3: SSE-S3; DynamoDB: AWS-managed encryption	NFR001

Control	Implementation	Requirement addressed
Secrets management	All credentials stored in AWS Secrets Manager; rotated automatically	NFR001
Network isolation	ECS services run in VPC private subnets; no public IP; NAT Gateway for outbound	NFR001
DDoS protection	AWS Shield Standard + WAF managed rule groups	NFR001
IAM least privilege	Each ECS task role has minimal permissions; no wildcard policies	NFR001
Input validation	Bean Validation (Jakarta) in Spring Boot; OWASP rules in WAF	NFR001


Edge: DDoS + rate limiting
Shield Standard + WAF · 100/300 req/min
Transport: TLS 1.2+
ACM certs on CloudFront + ALB
API: JWT validation
Gateway validates every request
Service: RBAC
@PreAuthorize · role in JWT
Application: input validation
Bean Validation · WAF OWASP
Data: encryption at rest
RDS KMS · S3 SSE · Dynamo
Secrets Manager
Auto-rotate · no hardcoding
BCrypt · cost factor 12
RBAC permissions
Admin
Full API access
User CRUD · course approval
Log viewing
Grade distribution reports
JWT expiry: 15 min / 7 days
Instructor
Own courses only
Create · publish · archive
Grade submissions
View enrolled students
Post announcements
Student
View published courses
Enrol · access content
Submit (within MaxAttempts)
Own progress + grades
Notification preferences
Network: VPC private subnet

7. Performance and Scalability

7.1 Performance targets

Metric	Target	Design decision
Course content load time	≤ 30 seconds (NFR001)	CloudFront CDN + S3 pre-signed URLs bypass the API layer for video streaming
API response time (P95)	< 500ms	ElastiCache Redis caching for metadata (10-min TTL); read replicas on RDS
Quiz auto-submit	On timer expiry	Client-side countdown with server-side timestamp validation; EventBridge backup trigger
Real-time notifications	< 2 seconds end-to-end	API Gateway WebSocket; SNS fanout to notification service
Certificate generation	< 30 seconds after completion	Async Lambda triggered by Progress service event; PDF stored in S3

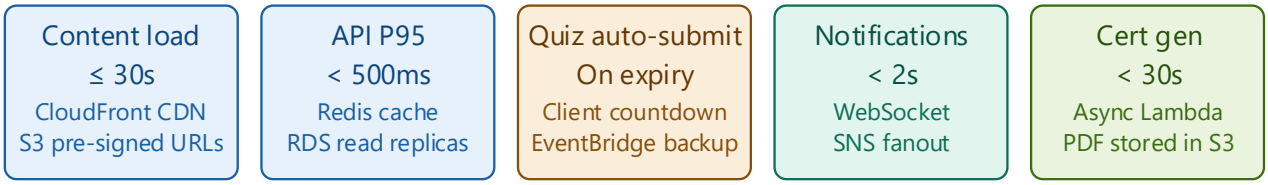
7.2 Scalability approach

- ECS Fargate services: auto-scaling based on CPU/memory metrics via Application Auto Scaling
- RDS: Multi-AZ for HA; read replicas for reporting queries; connection pooling via HikariCP
- DynamoDB: on-demand capacity mode — scales automatically with write spikes from lesson views and logs
- SQS: absorbs submission and log write bursts; Lambda consumers scale concurrently
- S3 + CloudFront: infinite horizontal scale for content delivery; no origin bottleneck for video streaming
- ElastiCache: cluster mode with sharding for high cache read throughput

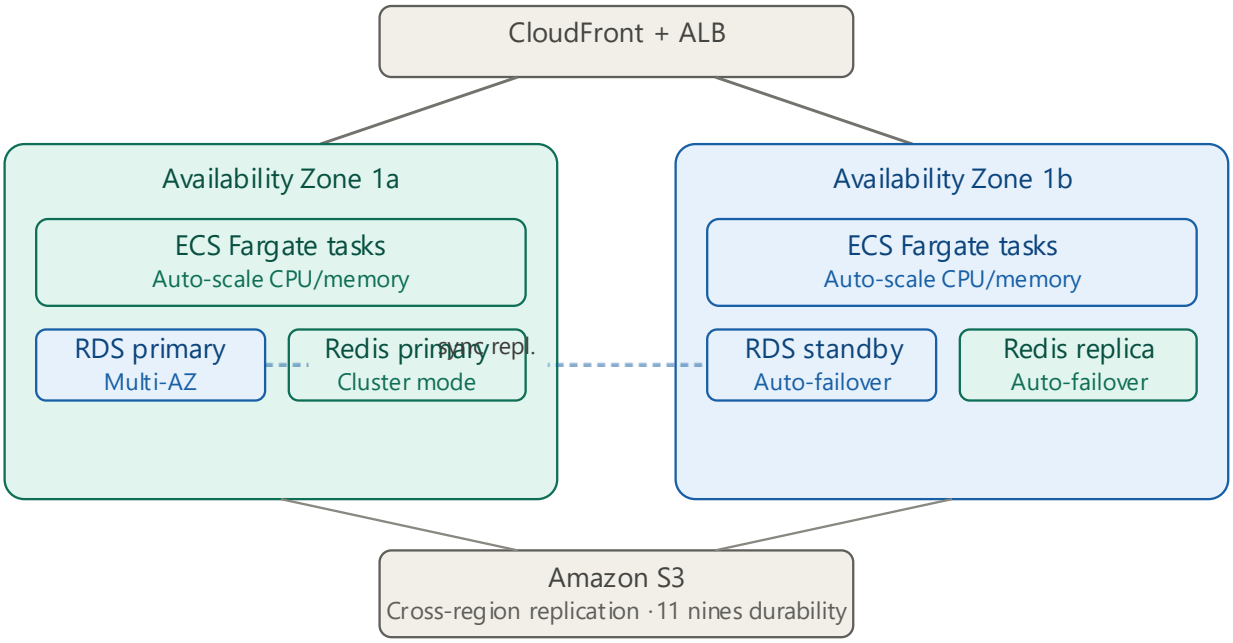
7.3 High availability

- All ECS services deployed across 2 Availability Zones minimum
- RDS Multi-AZ with automatic failover
- ElastiCache Multi-AZ replication group
- S3 cross-region replication for content durability
- ALB health checks with automatic unhealthy instance replacement

- CloudFront origin failover groups for edge resilience



High availability — 2 availability zones



8. CI/CD and DevOps

8.1 Pipeline stages

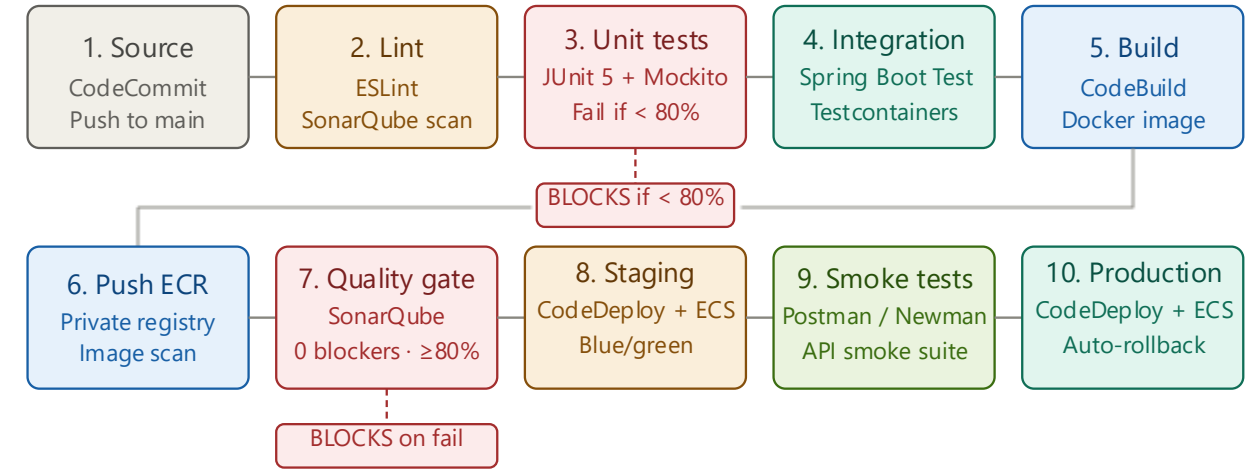
Stage	Tool	Action
Source	AWS CodeCommit / GitHub	Push to main branch triggers pipeline
Static analysis	ESLint + SonarQube	Lint checks; fail build on error-level violations
Unit tests	JUnit 5 + Mockito	Run unit tests; fail if coverage < 80% (F011)
Integration tests	Spring Boot Test + Testcontainers	Test API endpoints against real DB containers
Build	AWS CodeBuild + Docker	Build Docker image; tag with Git SHA
Image push	Amazon ECR	Push image to private ECR repository
Quality gate	SonarQube	Block promotion if quality gate fails
Deploy to staging	AWS CodeDeploy + ECS	Blue/green deployment to staging environment
Smoke tests	Postman / Newman	Run API smoke tests against staging
Deploy to production	AWS CodeDeploy + ECS	Blue/green deployment; automatic rollback on health check failure

8.2 Code quality standards

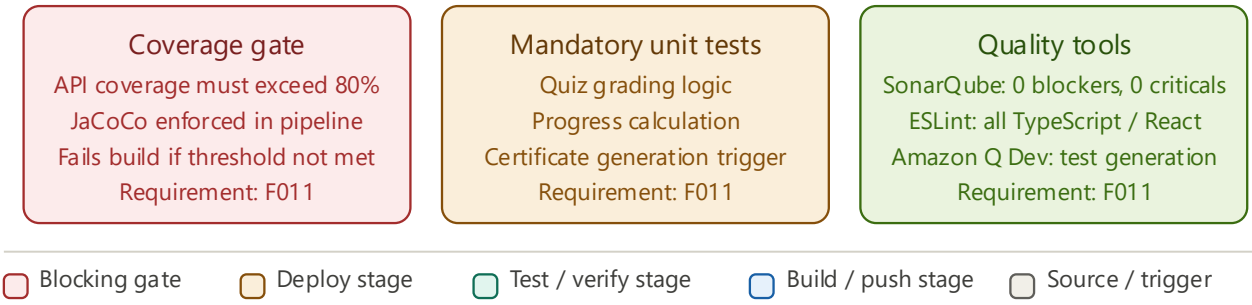
- API code coverage must exceed 80% (F011)
- Unit tests are mandatory for: quiz grading logic, progress calculation, certificate generation trigger (F011)
- SonarQube quality gate: 0 blocker issues, 0 critical issues, coverage ≥ 80%
- ESLint enforced on all TypeScript/JavaScript code in the React frontend
- Amazon Q Developer (GenAI code companion) used to accelerate unit test generation

Pipeline stages

Push to main branch triggers all 10 stages automatically

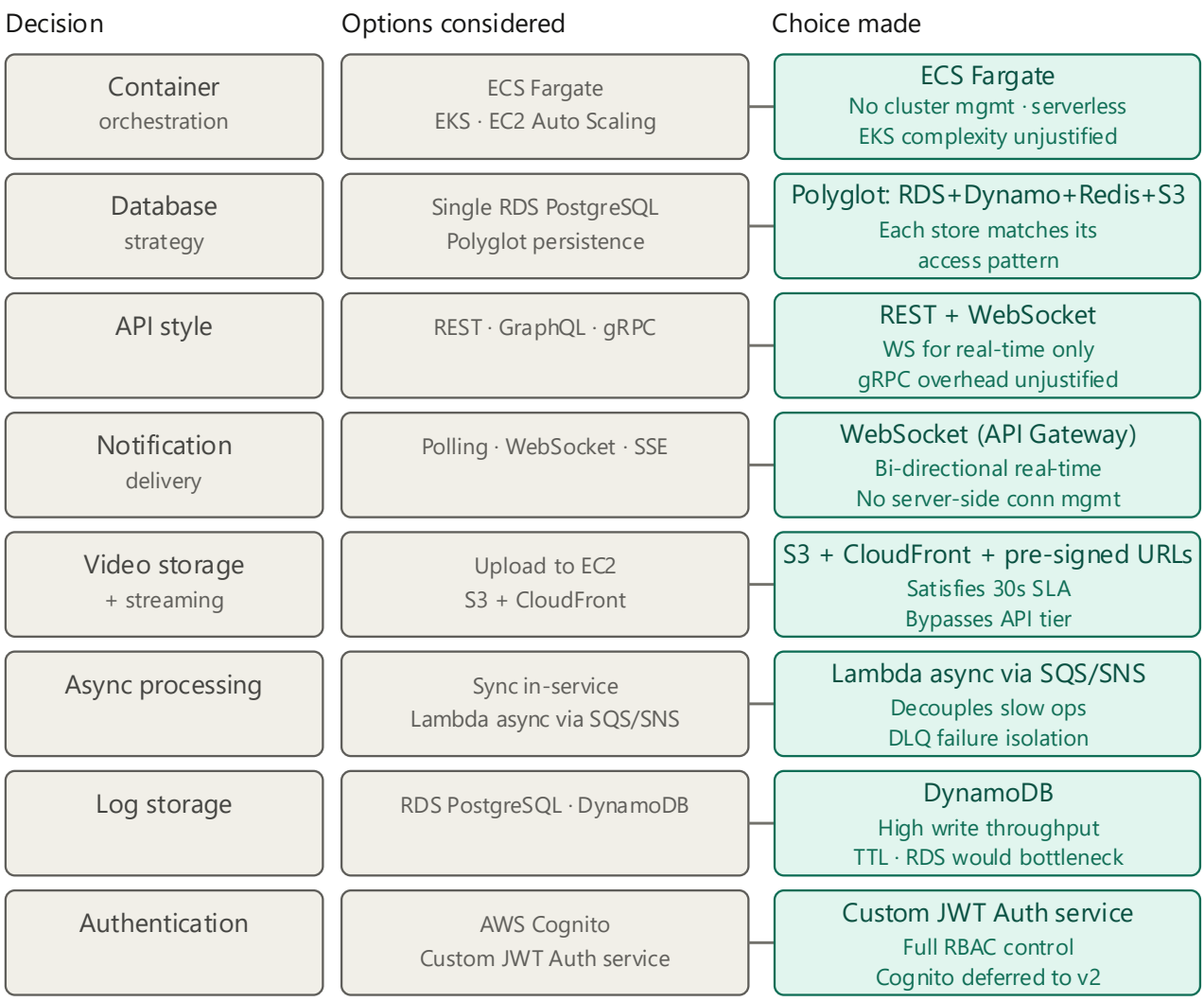


Code quality standards (F011)



9. Key Architecture Design Decisions

This section documents the key design decisions made during the architecture process, including the options considered and the rationale for each choice. These are the decisions the ARB should evaluate.



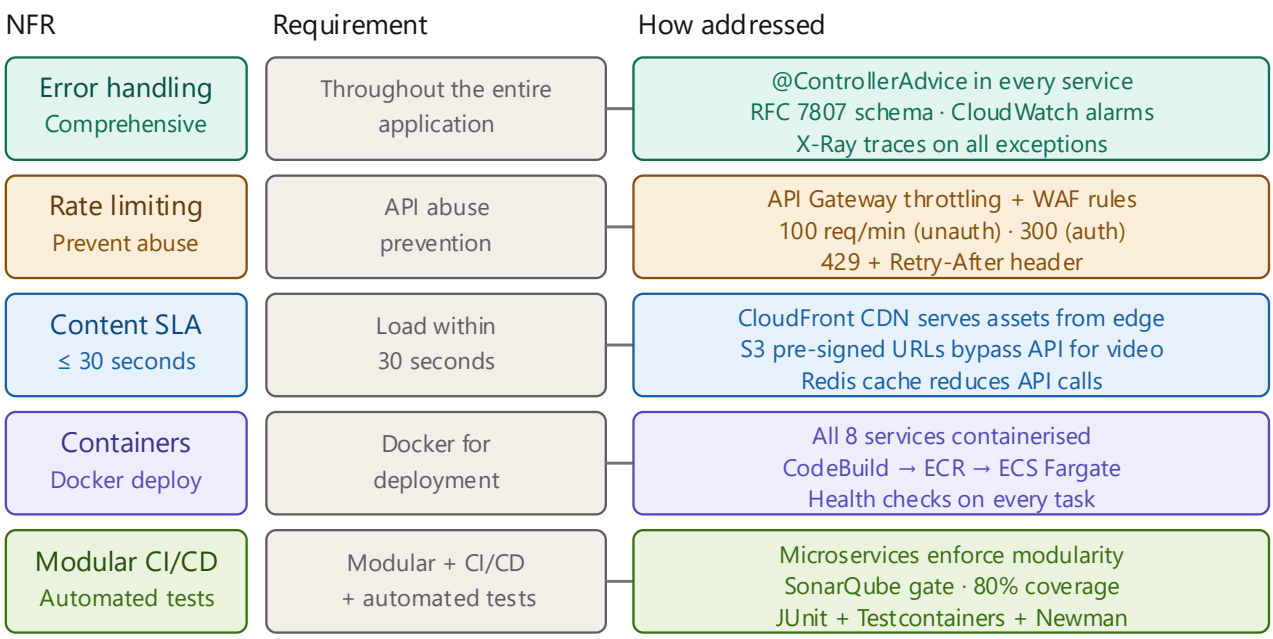
Decision	Options considered	Decision made	Rationale
Container orchestration	ECS Fargate vs EKS (Kubernetes) vs EC2	ECS Fargate	Lower operational overhead; no cluster management; serverless containers align with team scale; EKS adds complexity not justified at launch
Database strategy	Single RDS PostgreSQL vs Polyglot persistence	Polyglot: RDS + DynamoDB + Redis + S3	Different data types have different access patterns — relational for ACID transactions, DynamoDB for high-write logs,

Decision	Options considered	Decision made	Rationale
			Redis for low-latency cache, S3 for binary objects
API style	REST vs GraphQL vs gRPC	REST + WebSocket	REST is well-understood, tooling-rich, and sufficient for the access patterns; WebSocket added specifically for real-time notifications only; gRPC overhead not justified
Notification delivery	Polling vs WebSocket vs SSE	WebSocket (API Gateway)	Bi-directional, real-time; API Gateway managed WebSocket removes server-side connection management; polling rejected due to auto-refresh latency and server load
Video storage and streaming	Upload to EC2 vs S3+CloudFront	S3 + CloudFront + pre-signed URLs	Satisfies 30s load SLA (NFR001); pre-signed URLs bypass API tier for direct browser-to-CDN streaming; S3 infinite scale; no egress through application servers
Async processing	Synchronous in-service vs Lambda async	Lambda async via SQS/SNS	Decouples slow operations (grading, cert gen) from request path; improves API response time; Lambda scales independently; failure isolation via DLQ
Log storage	RDS PostgreSQL vs DynamoDB	DynamoDB	High write throughput (every lesson view fires a log entry); flexible schema; no table locking; TTL for automatic log expiry; RDS would become a write bottleneck

Decision	Options considered	Decision made	Rationale
Authentication	AWS Cognito vs custom JWT Auth service	Custom Spring Boot Auth service with JWT	Full control over RBAC logic and token claims; Cognito noted as optional enhancement for OAuth 2.0 / social login in a future phase

10. Non-Functional Requirements Compliance

NFR	Requirement	How it is addressed
Error handling	Comprehensive throughout application	Global exception handler in each Spring Boot service (@ControllerAdvice); standardised error response schema; CloudWatch Alarms on error rate thresholds; X-Ray traces on all exceptions
Rate limiting	Prevent API abuse	API Gateway throttling (configurable burst and steady-state limits per endpoint); AWS WAF rate-based rules per IP; 429 responses with Retry-After header
Content load SLA	Load within 30 seconds	CloudFront CDN serves static assets from edge; S3 pre-signed URLs for direct video streaming; ElastiCache Redis metadata cache reduces API calls; CloudFront measured P95 < 2s for assets
Containerisation	Docker for deployment	All Spring Boot services containerised; Docker images built in CodeBuild; stored in ECR; deployed to ECS Fargate with health checks
Modular codebase	Modular with CI/CD and automated tests	Microservices architecture enforces modularity; each service has independent pipeline; SonarQube quality gate; 80% coverage gate; JUnit + Testcontainers; Postman smoke tests



11. Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
Video streaming exceeds 30s SLA on slow connections	Medium	High	CloudFront + adaptive bitrate streaming; HLS/DASH segmentation; fallback quality tiers
DynamoDB hot partition on high-traffic courses	Low	Medium	Design partition keys to distribute writes (e.g. actorId + date prefix); DynamoDB on-demand mode absorbs spikes
JWT token compromise	Low	High	Short access token TTL (15 min); refresh token rotation; token blacklist in Redis on logout
Manual grading SLA breach	Medium	Medium	Instructor dashboard alerts on ungraded submissions > 48hrs; Admin escalation report
SonarQube gate blocking releases	Medium	Low	Gate configured as warning (not blocker) for first 2 sprints while test coverage is built up; hard gate after sprint 3

Risk	Likelihood	Impact	Mitigation
ECS Fargate cold start latency	Low	Low	Keep minimum 1 task running per service; pre-warm via scheduled EventBridge ping if needed

1 — Video SLA breach (Med/High)
CloudFront + HLS/DASH adaptive bitrate
Fallback quality tiers

2 — DynamoDB hot partition (Low/Med)
Partition key: actorId + date prefix
On-demand mode absorbs spikes

3 — JWT token compromise (Low/Crit)
15-min access TTL · refresh token rotation
Redis blacklist on logout

4 — Manual grading SLA breach (Med/Med)
Instructor alert: ungraded > 48 hrs
Admin escalation report

5 — ECS cold start (Low/Low)
Keep min 1 task running per service
EventBridge pre-warm ping

6 — SonarQube gate blocking (Med/Low)
Warning-only for sprints 1–2
Hard gate enforced from sprint 3

12. Glossary

Term	Definition
ARB	Architecture Review Board — governance body responsible for approving architectural decisions
CDN	Content Delivery Network — geographically distributed network of proxy servers that deliver content from edge locations close to users
ECS Fargate	Amazon Elastic Container Service with Fargate launch type — serverless container orchestration without managing EC2 instances
HLD	High Level Design — architecture document describing system structure, component interactions, and key design decisions
JWT	JSON Web Token — compact, URL-safe means of representing claims transferred between two parties for authentication

Term	Definition
LLD	Low Level Design — detailed design document covering class diagrams, sequence diagrams, API contracts, and data models
LMS	Learning Management System — software platform for creating, delivering, and managing educational content and assessments
RBAC	Role-Based Access Control — access control model where permissions are assigned to roles, and users are assigned to roles
SLA	Service Level Agreement — defined performance target, e.g. content must load within 30 seconds
SNS	Amazon Simple Notification Service — managed pub/sub messaging for fanout of events to multiple subscribers
SQS	Amazon Simple Queue Service — managed message queue for decoupling and buffering asynchronous workloads
VPC	Virtual Private Cloud — logically isolated section of the AWS Cloud where resources run in a defined virtual network